Objektno orijentirano programiranje
Vježba 6

Napomena za sve zadatke: data memberi su `private`. Osim navedenih metoda, klase smiju imati i dodatne: get metode za čitanje članova, pomoćne privatne metode i set metode koje čuvaju invarijantu klase (npr. provjerom ili ograničavanjem vrijednosti).

1. Napišite klasu IntStack: stog cijelih brojeva koji elemente čuva u nizu na heapu.
   - Članovi: pointer na niz, broj elemenata i veličina niza (kapacitet). Konstruktor prima početni kapacitet (zadano 2) i zauzima niz s `new[]`; destruktor ga oslobađa.
   - `push(int x)`: ako je niz pun, zauzmite novi niz dvostruke veličine, prepišite elemente, oslobodite stari niz i tek onda dodajte x.
   - `pop()`, `top()`, `empty()`, `size()` i `capacity()`. `capacity()` vraća veličinu niza
   - Napišite copy konstruktor (deep copy).

   Program za testiranje je u datoteci `intstack_main.cpp`.
   Što bi se ispisalo bez vlastitog copy konstruktora?

2. Proširite klasu Ticket iz prošle vježbe.
   - Napišite `Ticket t4 = t1;` i ispišite brojeve obiju ulaznica. Zašto rezultat nije ispravan?
   - Ulaznica se ne smije duplicirati: zabranite kopiranje (`= delete` za copy konstruktor). Provjerite da se `Ticket t4 = t1;` više ne prevodi.
   - Napišite funkciju `void print(const Ticket& t)` i pozovite je s `print(t1)`. Zatim promijenite parametar u `Ticket t`. Zašto se poziv više ne prevodi?

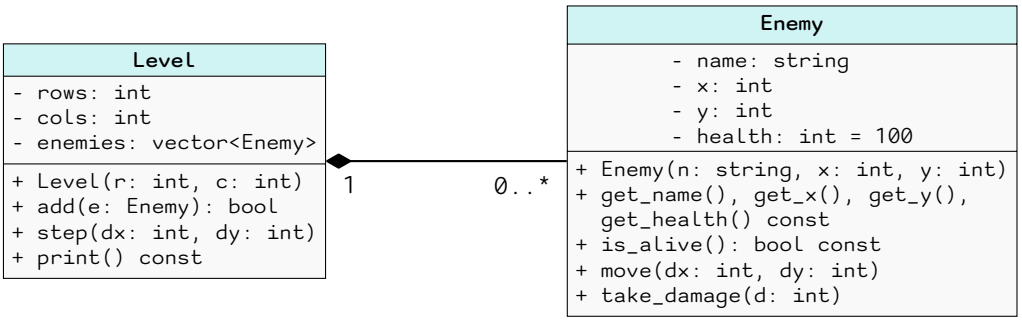
3. Napišite klasu LogFile koja u konstruktoru otvara datoteku za dodavanje na kraj, a u destruktoru je zatvara. Metoda `write(const std::string& line)` zapisuje jedan redak. Kopiranje zabranite. Za rad s datotekom koriste se funkcije iz C-a (`#include <cstdio>`):
   - `std::FILE* f = std::fopen(name.c_str(), "a");` otvara datoteku za dodavanje na kraj; vraća `nullptr` ako otvaranje ne uspije;
   - `std::fputs(s.c_str(), f);` zapisuje string s (bez novog retka, `"\n"` treba dodati);
   - `std::fclose(f);` zatvara datoteku.

   Metoda `c_str()` je potrebna jer funkcije iz C-a primaju `const char*`, a ne `std::string`. Napišite funkciju `int log_songs(const std::vector<std::string>& titles)` koja zapisuje naslove pjesama u datoteku `played.log`, a kad naiđe na `"STOP"`, odmah vraća broj zapisanih naslova. Program za testiranje je u datoteci `logfile_main.cpp`.

4. Napišite klasu Board: ploču znakova dimenzija zadanih u konstruktoru, s rubom od znaka zadanog u konstruktoru. Znakovi se čuvaju u jednom nizu na heapu (`rows * cols` elemenata, element `(i, j)` je na indeksu `i * cols + j`).
   - Metoda `set(i, j, ch)` postavlja znak, ali samo unutar ruba; `get(i, j)` vraća znak; `rows()` i `cols()` vraćaju dimenzije; `print()` ispisuje ploču.
   - Napišite destruktor i copy konstruktor (deep copy).

   Program za testiranje je u datoteci `board_main.cpp`.
   Dodatno: Napišite drugu verziju klase Board u kojoj su znakovi u `std::vector<char>` od `rows * cols` elemenata. Program za testiranje je isti. Koje metode iz prve verzije više nisu potrebne i zašto?

5. Prema dijagramu klasa napišite klase Enemy i Level. Level je pravokutnik od rows redaka i cols stupaca; neprijatelj je na polju (x, y), gdje je x stupac (0 do cols - 1), a y redak (0 do rows - 1).



   - Metoda take_damage smanjuje zdravlje, ali ne ispod 0. Neprijatelj je živ dok mu je zdravlje veće od 0.
   - Level na početku nema neprijatelja; dodaju se metodom add.
   - Ako položaj neprijatelja nije unutar granica levela, metoda add neprijatelja ne dodaje u vektor i vraća false, inače ga dodaje i vraća true.
   - Metoda step(dx, dy) pomiče svakog neprijatelja za (dx, dy). Neprijatelj koji bi izašao iz levela ostaje na mjestu.
   - Metoda print() ispisuje svakog neprijatelja u obliku ime (x, y) zdravlje.

   Program za testiranje je u datoteci level_main.cpp.
   Dodatno: metoda draw() const koja napravi Board iz prethodnog zadatka dimenzija (rows + 2) x (cols + 2) s rubom '#', na mjesto svakog neprijatelja upiše prvo slovo njegovog imena i ispiše ploču.

6. Aplikacija za glazbu pamti zadnjih N reproduciranih pjesama (”nedavno slušano”). Kad je popis pun, nova pjesma zamjenjuje najstariju. Napišite klasu RecentlyPlayed koja naslove čuva u **kružnom bufferu**: nizu fiksne veličine na heapu u kojem se indeks za upis vrti u krug (iza zadnjeg mjesta dolazi prvo).

   - Članovi: pointer na niz stringova, kapacitet, broj spremljenih naslova i indeks na koji ide sljedeći upis. Konstruktor prima kapacitet i zauzima niz s new[]; kapacitet mora biti veći od 0 (provjerite s assert). Destruktor oslobađa niz.
   - Metoda play(title) upisuje naslov na sljedeće mjesto i pomiče indeks za jedno mjesto, a s kraja niza na početak (operator %). Kad je buffer pun, tako se prepisuje najstariji naslov.
   - Metode size() i capacity() vraćaju broj spremljenih naslova i kapacitet.
   - Metoda get(i) vraća i-ti naslov od najnovijeg (get(0) je zadnja reproducirana pjesma).
   - Metoda print() ispisuje naslove od najnovijeg prema najstarijem, odvojene zarezom.
   - Napišite copy konstruktor (deep copy).

   Program za testiranje je u datoteci recent_main.cpp.